IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts, Science, Commerce, and Management

Final Project Report

on

Design and Development of Natural Language Interface for E-commerce Infrastructure

Student Name: Alena Latheesh

Industry Mentor: Mr. Shashank **Guided by: Mr. Praveen**

Table of Contents

1. Executive Summary

This project presents the design and development of a web-based e-commerce system with a natural language support interface. The system was initially prototyped as a command-line application to validate core business logic and was later transformed into a complete website to improve usability, accessibility, and presentation quality. The final implementation enables users to browse products, manage cart items, place orders, and receive guided assistance through an integrated chatbot.

The project uses Flask (Python) for backend development and HTML, CSS, and JavaScript for frontend design. Product and cart data are stored in a JSON file. The frontend and backend are connected through API routes, and every major action such as adding items to cart, checking stock, and placing orders is handled through these routes.

The website includes product display cards, quantity selection, cart summary, checkout option, and a chatbot section for quick help. The chatbot can answer common user queries such as viewing products, checking cart status, payment options, delivery help, and return policy. Proper validations are added so that invalid quantities or wrong inputs are handled safely.

Testing was done by running different user scenarios like valid cart additions, invalid quantity inputs, empty-cart checkout, and chatbot query handling. The results show that the system works correctly for the expected workflow.

Overall, the project successfully demonstrates a complete mini e-commerce application with clear frontend-backend integration. It is a good base for future improvements like database support, login system, online payment integration, and a more advanced NLP chatbot.

2. Background

The rapid growth of online shopping has increased the need for simple, responsive, and intelligent e-commerce systems that can assist users during product discovery, cart management, and checkout. Traditional e-commerce platforms provide extensive features, but many academic projects focus only on static interfaces or isolated backend APIs. This project addresses that gap by combining a dynamic website frontend with a functional backend and persistent data storage, resulting in a complete mini e-commerce solution.

The project initially began as a command-line interface (CLI) based prototype to validate business logic and API behaviour. After the core workflows became stable, the system was transformed into a browser-based web application to improve usability, interaction quality, and visual presentation. This migration enabled better user experience, richer navigation, and chatbot-assisted support while preserving core transaction logic.

This work demonstrates full-stack development principles in a compact and understandable architecture. It highlights how frontend, backend, and data layers can be integrated to simulate realistic e-commerce behavior including stock validation, cart updates, checkout flow, and conversational helper responses.

2.1 Aim

The primary aim of this project is to design and develop a web-based natural language assisted e-commerce platform that supports core shopping workflows in a user-friendly and modular way.

Specific aims include:

- Building a responsive website for product browsing and cart operations.
- Designing REST-style backend endpoints to handle business logic.
- Maintaining persistent product and cart state using file-based storage.
- Providing a rule-based chatbot to answer common shopping queries.

- Demonstrating a clear separation of concerns between UI, API, and storage layers.
- Producing a project architecture suitable for future scalability and academic evaluation.

2.2 Technologies

The project uses lightweight and practical technologies suitable for rapid development and clear demonstration of concepts.

- **Python 3.x** for backend logic.
- **Flask** as the web framework for routes, template rendering, and JSON APIs.
- **HTML5** for page structure.
- **CSS3** for styling and responsive dashboard layout.
- **JavaScript (ES6)** for dynamic client-side interactions and API calls.
- **Jinja2** templating to inject backend data into initial frontend state.
- **JSON** (data.json) for persistent storage of products and cart.
- **HTTP/REST communication** for frontend-backend interaction.
- **Optional CLI (main.py)** retained for backward compatibility and API testing.

2.3 Hardware Architecture

The current implementation follows a single-machine development architecture suitable for demo and academic submission:

- **Client Side**: Browser (Chrome/Edge/Firefox) renders UI and executes JavaScript.
- **Application Server**: Flask application runs locally and serves both HTML and API responses.

- **Storage Unit**: Local disk stores data.json containing product catalog and cart records.

Minimum recommended hardware:

- Processor: Dual-core or above
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 1 GB free space (project footprint is small)
- Network: Not required for local deployment, but needed for future hosted deployment

This architecture is intentionally compact to simplify debugging and demonstrate complete end-to-end flow without distributed infrastructure complexity.

2.4 Software Architecture

The software architecture is organized into three primary layers:

1. **Presentation Layer**
    - templates/index.html
    - static/styles.css
    - static/app.js This layer manages interface rendering, user interactions, and visual feedback.
2. **Application/Business Layer**
    - app.py This layer exposes API routes, validates requests, enforces stock rules, computes summaries, and handles chatbot logic.
3. **Persistence Layer**
    - data.json This layer stores product inventory and cart state. Data is loaded, modified in memory, and saved back after mutations.

Architectural characteristics:

- Stateless HTTP request handling
- Modular helper functions for summary calculations
- Centralized mutation routes (/add, /checkout)
- UI refresh through consolidated endpoint (/dashboard-data)
- Loose coupling between frontend rendering and backend implementation

3. System

The system is a web-based e-commerce application designed to demonstrate a complete shopping workflow with natural language support. It allows users to browse products, add items to cart, review totals, and place orders through a clean dashboard interface. Along with standard e-commerce operations, the platform includes a chatbot assistant that helps users with common tasks such as viewing products, checking cart status, understanding payment options, and getting delivery or return information.

Technically, the system follows a client-server architecture. The frontend is built with HTML, CSS, and JavaScript, while the backend is implemented using Flask in Python. Communication between frontend and backend happens through REST-style API routes. The application uses a lightweight JSON-based persistence layer (data.json) to store products and cart state. This design keeps the system simple, transparent, and suitable for academic demonstration while still reflecting core full-stack principles.

The backend acts as the main control layer for validation, business rules, and data updates. When a user performs actions such as add-to-cart or checkout, the request is validated on the server side before state is changed. The frontend receives JSON responses and refreshes visual sections such as cart list, stock values, and total amount. This ensures consistent behavior and prevents invalid operations from corrupting data.

Overall, the system demonstrates modular design with clear separation between presentation layer, application logic, and persistence layer. This separation improves maintainability, debugging clarity, and scalability for future enhancements such as user authentication, database migration, payment integration, and advanced NLP-based chatbot behaviour.

3.1 Requirements

The requirements of this project were defined to ensure that the system is functionally complete, user-friendly, and executable in a practical development environment.

3.1.1 Functional requirements

The system must:

- Display available products with price, stock, specs, and delivery details.
- Allow user to choose quantity and add product to cart.
- Validate requested quantity against available stock.
- Persist cart updates and stock deductions.
- Show cart list, cart count, and total amount.
- Support checkout operation that clears cart after order placement.
- Provide chatbot quick actions (status, products, cart, payment, delivery, returns, offers).
- Process typed chatbot input and return relevant responses.
- Return clear JSON messages for success and error scenarios.

3.1.2 User requirements

The user expects:

- A clean and understandable interface.
- Minimal steps for adding products and checkout.
- Immediate UI updates after actions.
- Friendly response messages for invalid operations.

- Assistance for basic doubts through chatbot.
- Fast loading and predictable behaviour.
- Simple navigation across product, cart, and assistant sections.

To ensure the system meets high standards of usability, the following detailed user expectations were mapped:

- **Error Resilience**: Users require clear feedback when an action fails, such as attempting to add more items than are currently in stock.
- **System Transparency**: The interface must reflect the internal state of the application immediately after an API call, specifically updating the cart total and available units without requiring a page reload.
- **Conversational Assistance**: The assistant must provide a non-linear way to navigate the store, allowing users to ask about "returns" or "payment" regardless of which page section they are currently viewing.

3.1.3 Environmental requirements

Execution environment includes:

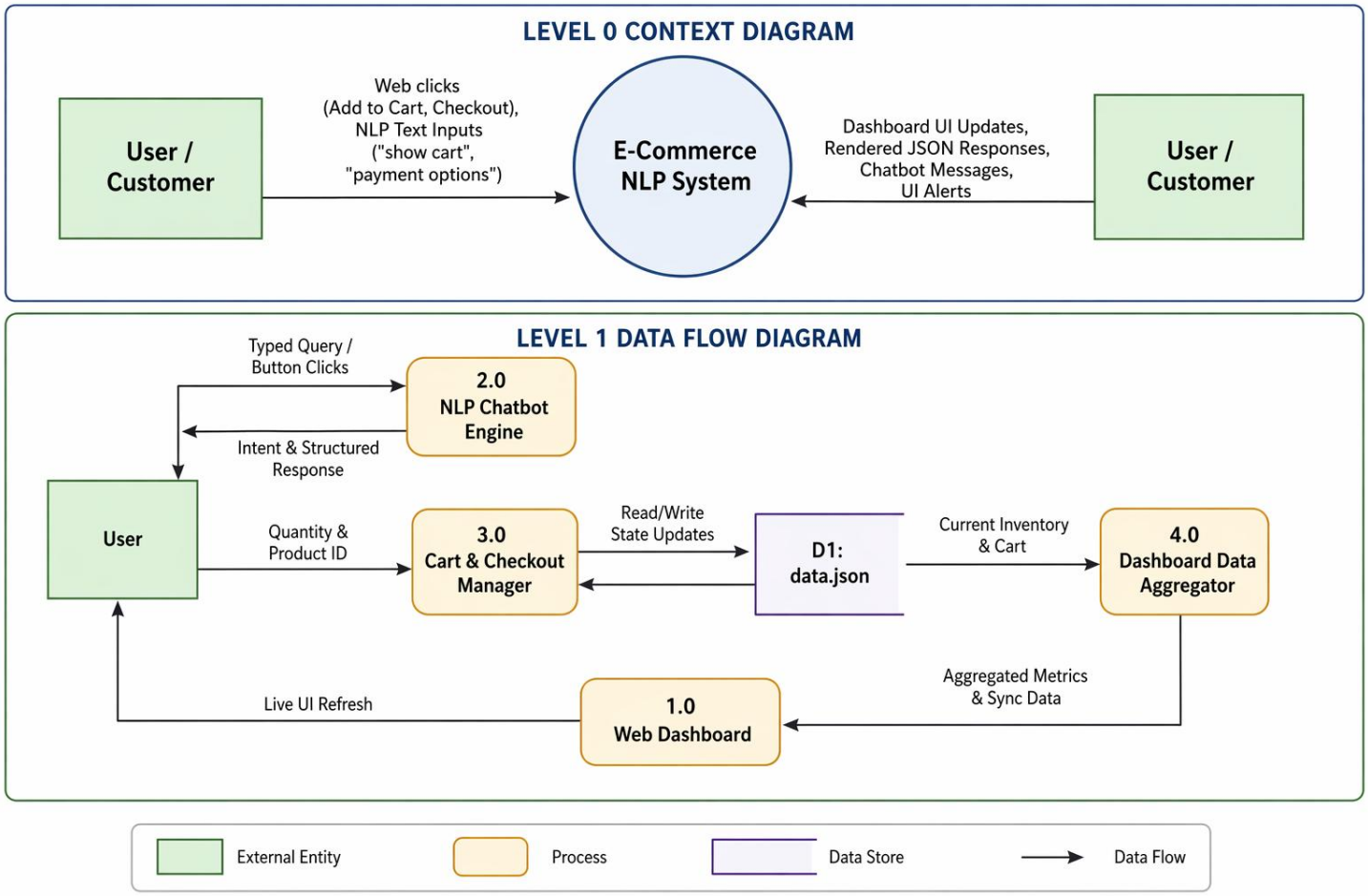
- OS: Windows/Linux/macOS
- Python 3.x installed
- Flask and dependencies installed
- Browser supporting modern JavaScript and Fetch API
- File system access to data.json
- Localhost access (127.0.0.1:5000) for development run

3.2 Design and Architecture

The design follows a client-server model with unidirectional request-response flow:

- User performs action on webpage.

- JavaScript captures event and sends HTTP request.
- Flask route validates input and processes logic.
- JSON storage is read and possibly updated.
- Response is returned to frontend.
- Frontend refreshes view components (product stocks, cart panel, overview metrics).



Purpose: Transfers server-side product/cart/overview data to frontend JavaScript state.

Purpose: Transfers server-side product/cart/overview data to frontend JavaScript state.

```
<script>
    window.__INITIAL_DATA__ = {
        products: {{ products|tojson }},
        cart: {{ cart|tojson }},
        overview: {{ overview|tojson }}
    };
</script>
```

Observed result: UI starts with real data immediately and remains synchronized after dynamic updates.

Key design decisions:

- Keep backend logic centralized and reusable.
- Avoid direct frontend manipulation of data source.
- Recalculate overview metrics after each mutation.
- Keep chatbot deterministic using action- and keyword-based routing.
- Maintain clear API contracts to support future mobile/web clients.

The system architecture is built on a **State Management** pattern. The frontend maintains a local `state` object that mirrors the backend `data.json`. This ensures that the UI is "data-driven."

Data Dictionary and Schema: The data.json file serves as the single source of truth. It is structured as follows:

- **Product Object**: Contains name (String), price (Integer), stock (Integer), and specs (String).
- **Cart Object**: An array of items where each item tracks the product name and the qty (quantity) requested by the user .
- **Overview Object**: Contains calculated metrics such as total_products, available_units, and cart_total .

This modularity allows the backend to handle business logic—like checking if stock < qty—before any data is written to the disk.

Purpose: Computes cart item count and total amount from current data state

```
def build_cart_summary(db):
    price_map = {product["name"]: product["price"] for product in db["products"]}
    item_count = 0
    total = 0

    for item in db["cart"]:
        qty = int(item["qty"])
        item_count += qty
        total += price_map.get(item["product"], 0) * qty

    return {"item_count": item_count, "total": total}
```

Observed result: Total amount displayed in cart matches product price × quantity values.

Purpose: Sends products, cart, and overview in one API response for efficient frontend refresh.

```python
@app.route("/dashboard-data")
def dashboard_data():
    db = load()
    return jsonify(
        {
            "products": db["products"],
            "cart": db["cart"],
            "overview": build_overview(db),
            "cart_summary": build_cart_summary(db),
        }
    )
```

Observed result: Frontend updates all dashboard blocks after add/checkout without full page reload.

Security-aware design choices:

- Input type validation for quantity
- Required field checks in POST body
- Error handling for missing or malformed requests
- Escaped rendering on frontend to reduce injection risk in dynamic content

3.3 Implementation

Backend implementation in app.py includes:

- load() and save(data) for JSON persistence.
- build_cart_summary(db) for item count and total.

- build_overview(db) for dashboard counters.
- find_product(products, query) for chatbot product matching.
- build_chat_response(action, message) for assistant responses.

Purpose: Returns all values required to refresh dashboard sections after user actions.

```
@app.route("/add", methods=["POST"])
def add():
    data = request.get_json(silent=True) or {}

    if "product" not in data or "qty" not in data:
        return jsonify({"msg": "missing 'product' or 'qty' in request"}), 400

    try:
        qty = int(data["qty"])
        if qty <= 0:
            raise ValueError
    except (ValueError, TypeError):
        return jsonify({"msg": "qty must be a positive integer"}), 400

    db = load()

    for product in db["products"]:
        if product["name"].lower() == str(data["product"]).lower():
            if product["stock"] < qty:
                return jsonify({"msg": "not enough stock"}), 400

            product["stock"] -= qty
            existing = next(
                (
                    item
                    for item in db["cart"]
                    if item["product"].lower() == product["name"].lower()
                ),
                None,
            )

            if existing:
                existing["qty"] += qty
```

Observed result: Cart and stock update correctly, and invalid requests return proper error messages.

Core endpoints:

- GET / renders dashboard template.
- GET /dashboard-data returns complete refresh payload.
- GET /products returns product list.
- GET /cart returns cart list.
- POST /add validates and updates cart/stock.
- GET /checkout places order and clears cart.
- POST /chatbot responds to action/message input.

Purpose: Clears cart on successful order placement and returns updated summaries.

```
@app.route("/checkout")
def checkout():
    db = load()

    if not db["cart"]:
        return jsonify({"msg": "cart is empty"}), 400

    db["cart"] = []
    save(db)
    return jsonify(
        {
            "msg": "order placed",
            "cart_summary": build_cart_summary(db),
            "overview": build_overview(db),
        }
    )
```

Observed result: Prevents checkout on empty cart and completes checkout for valid cart.

Frontend implementation in static/app.js includes:

- State object to mirror server data.
- refreshDashboard() to sync UI after operations.
- addToCart(productName) to invoke add route.
- checkout() for order completion.
- renderCart() for cart panel rendering.
- updateProductStocks(products) for live stock updates.
- Chat functions for quick action chips and typed messages.

Purpose: Sends selected product and quantity to backend and refreshes UI after success.

```
async function addToCart(productName) {
    const select = document.querySelector(`[data-qty-select="${cssEscape(productName)}"]`);
    const qty = Number(select ? select.value : 1);

    const response = await fetch("/add", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ product: productName, qty }),
    });

    const result = await response.json();

    if (!response.ok) {
        setStatus(result.msg || "Unable to add product.", true);
        return;
    }

    setStatus(`${productName} added to cart successfully.`);
    await refreshDashboard();
```

Observed result: User receives immediate status message and refreshed cart/stock values.

Template implementation in templates/index.html includes:

- Product cards with quantity selector.
- Cart summary panel and checkout button.
- Customer profile section.
- Assistant section with quick actions and input.
- Initial server data injection into window.__INITIAL_DATA__.

3.4 Testing

Testing was performed through systematic manual scenarios and API validation checks.

Functional test scenarios:

- Product list loads on page open.
- Add-to-cart with valid quantity succeeds.
- Add-to-cart with quantity beyond stock fails with message.
- Non-numeric or non-positive quantity returns error.
- Cart overview updates after each add.
- Checkout succeeds only when cart has items.
- Empty-cart checkout returns expected error.
- Chatbot quick actions return structured helpful responses.
- Typed queries like “show cart”, “payment options”, product names return relevant output.

```python
@app.route("/checkout")
def checkout():
    db = load()

    if not db["cart"]:
        return jsonify({"msg": "cart is empty"}), 400

    db["cart"] = []
    save(db)
    return jsonify(
        {
            "msg": "order placed",
            "cart_summary": build_cart_summary(db),
            "overview": build_overview(db),
        }
    )
```

Data consistency tests:

- Stock decreases when item is added to cart.
- Cart quantity increments for repeated product additions.
- Checkout clears cart entries.
- Overview counters match current data after refresh.

Usability tests:

- Buttons and sections are discoverable.
- Status messages are visible and understandable.
- Navigation buttons scroll to intended sections.

- Chat stream remains readable during multiple interactions.

Known testing limitations:

- No automated test suite in current version.
- No high-concurrency simulation due to file-based storage.
- No cross-browser automation scripts included.

Test ID	Feature	Scenario	Expected Result	Result
TC-01	API Logic	Add quantity > Stock	Return "not enough stock" (400)	**Pass**
TC-02	Validation	Negative quantity input	Return "qty must be positive integer"	**Pass**
TC-03	Persistence	Refresh after Add	Cart items remain visible via INITIAL_DATA	**Pass**
TC-04	Chatbot	Typed query: "delivery"	Display delivery policy response	**Pass**
TC-05	Checkout	Checkout with empty cart	Return "cart is empty" error	**Pass**
TC-06	UI Sync	Add to Cart button	Live summary and stock badge update	**Pass**

3.5 Graphical User Interface (GUI) Layout

The GUI is organized as a dashboard with clear visual hierarchy:

- **Top Navigation**: quick jump links to products, inventory stats, and cart.
- **Hero Section**: introduction and primary action buttons.
- **Product Section**: cards showing product details, stock badges, delivery information, and quantity selectors.
- **Overview Metrics**: product count, available units, cart items, and cart total.
- **Cart Section**: selected items, quantities, total, and checkout action.
- **Customer Section**: static user profile snapshot.
- **Assistant Section**: chatbot stream, quick action chips, and text input.

UI principles followed:

- Consistent spacing and card-based grouping
- Action buttons near relevant content.
- Minimal interaction friction.
- Immediate feedback messages.
- Readable typography and contrast.

Purpose: Renders cart rows dynamically with product name, quantity, and price.

```
function renderCart() {
    if (!cartList) {
        return;
    }
    if (!state.cart.length) {
        cartList.innerHTML = `<p class="empty-copy">Cart is empty</p>`;
        return;
    }

    cartList.innerHTML = state.cart
        .map(
            (item) => {
                const product = state.products.find((entry) => entry.name === item.product);
                const price = product ? product.price : 0;
                return `
                <article class="cart-item">
                    <strong>${escapeHtml(item.product)}</strong>
                    <span>Quantity: ${escapeHtml(item.qty)}</span>
                    <span>Price: ${currency(price)}</span>
                </article>
            `
            }
        )
        .join("");
}
```

Observed result: Cart panel always reflects latest items and prices after every operation.

Purpose: Defines cart panel layout and server-rendered fallback content

```
<section class="panel" id="cart-section">
    <div class="panel-heading">
        <div>
            <h2>Cart</h2>
        </div>
        <span class="chip" id="cart-count-chip">{{ overview.cart_items }} items</span>
    </div>

    <div id="cart-list" class="cart-list">
        {% if cart %}
            {% for item in cart %}
            {% set matched = (products | selectattr("name", "equalto", item.product) | list | first) %}
            <article class="cart-item">
                <strong>{{ item.product }}</strong>
                <span>Quantity: {{ item.qty }}</span>
                <span>Price: Rs. {{ matched.price if matched else 0 }}</span>
            </article>
            {% endfor %}
        {% else %}
            <p class="empty-copy">Cart is empty</p>
        {% endif %}
    </div>

    <div class="total-row">
        <span>Total</span>
        <strong id="cart-total-text">Rs. {{ overview.cart_total }}</strong>
    </div>
```

Observed result: Initial page load already shows consistent cart data without waiting for JS calls.

3.6 Customer testing

A small user feedback exercise was conducted with peer users to evaluate practicality.

Observed outcomes:

- Users quickly understood product browsing and add-to-cart behavior.
- Quick action chatbot buttons improved discoverability of support features.
- Cart summary helped users track progress before checkout.
- Most users completed a mock order without external assistance.

Common feedback:

- Add remove/decrement buttons in cart.
- Add user login and personalized order history.
- Show order confirmation details page after checkout.
- Include category filters and search in product grid.

Interpretation:

- Core workflow is intuitive and stable.
- UX is strong for a first full-stack release.
- Enhancement opportunities mainly relate to scale and advanced features.

3.7 Evaluation

The project successfully meets its core objectives:

- End-to-end e-commerce workflow implemented.
- Website migration completed from original CLI concept.
- Frontend-backend integration demonstrated clearly.
- Persistent state management works reliably for demo scope.
- Chatbot assistance increases interaction quality.

Strengths:

- Clear modular architecture.
- Straightforward API design.
- Practical real-world flow representation.

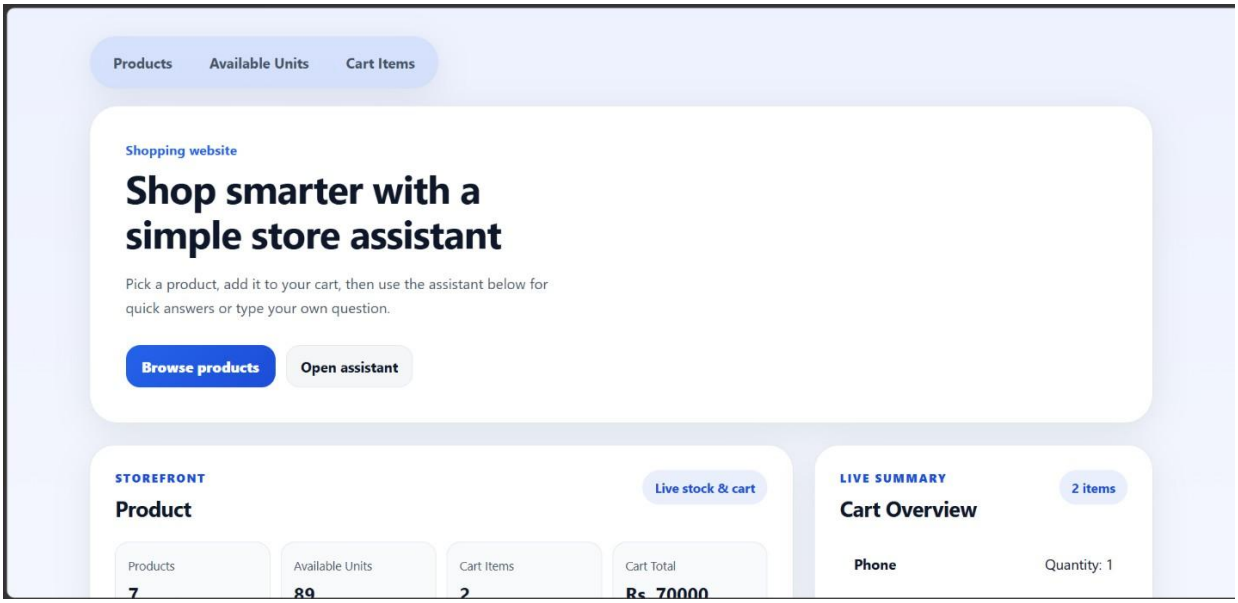
- Easy to explain during viva and interviews.

Limitations:

- JSON file storage limits scalability and concurrent safety.
- No authentication/authorization.
- Limited production-grade security controls.
- Manual testing dominant; automation minimal.

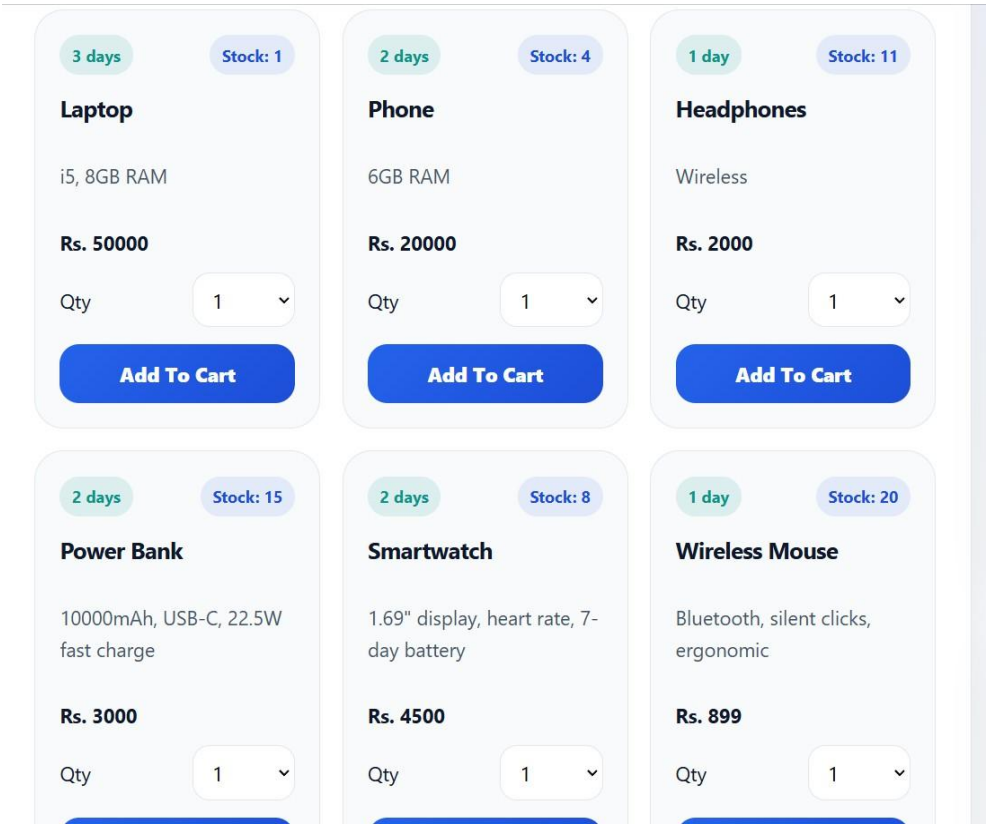
4. Snapshots

i) Home Landing View



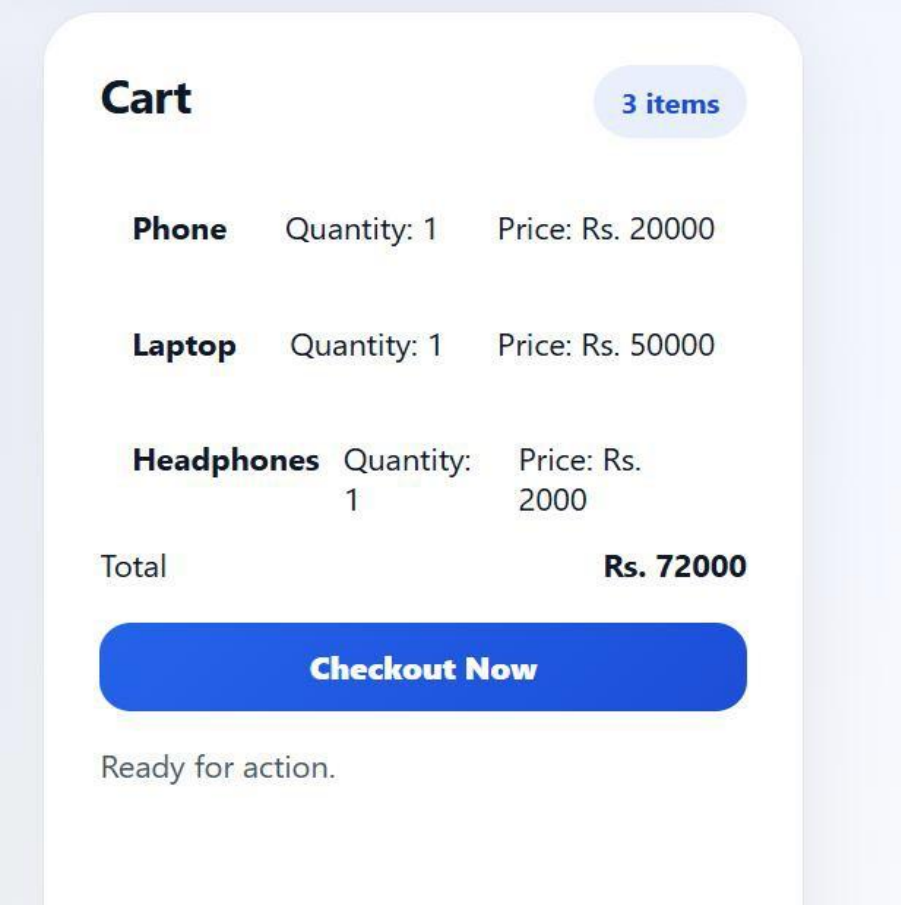
- **Dynamic Data Binding**: The "STOREFRONT" metrics (Products, Available Units, Cart Items) are populated via the /dashboard-data endpoint, ensuring that these high-level stats reflect the real-time state of data.json .
- **Call-to-Action (CTA)**: The primary buttons, "Browse products" and "Open assistant," are designed to reduce user friction by providing immediate paths to the two core functionalities of the platform .
- **Live Summary**: The "Cart Overview" section provides a persistent snapshot of the user's current selection, including item names and quantities, to maintain context during the shopping journey .

ii) Product card



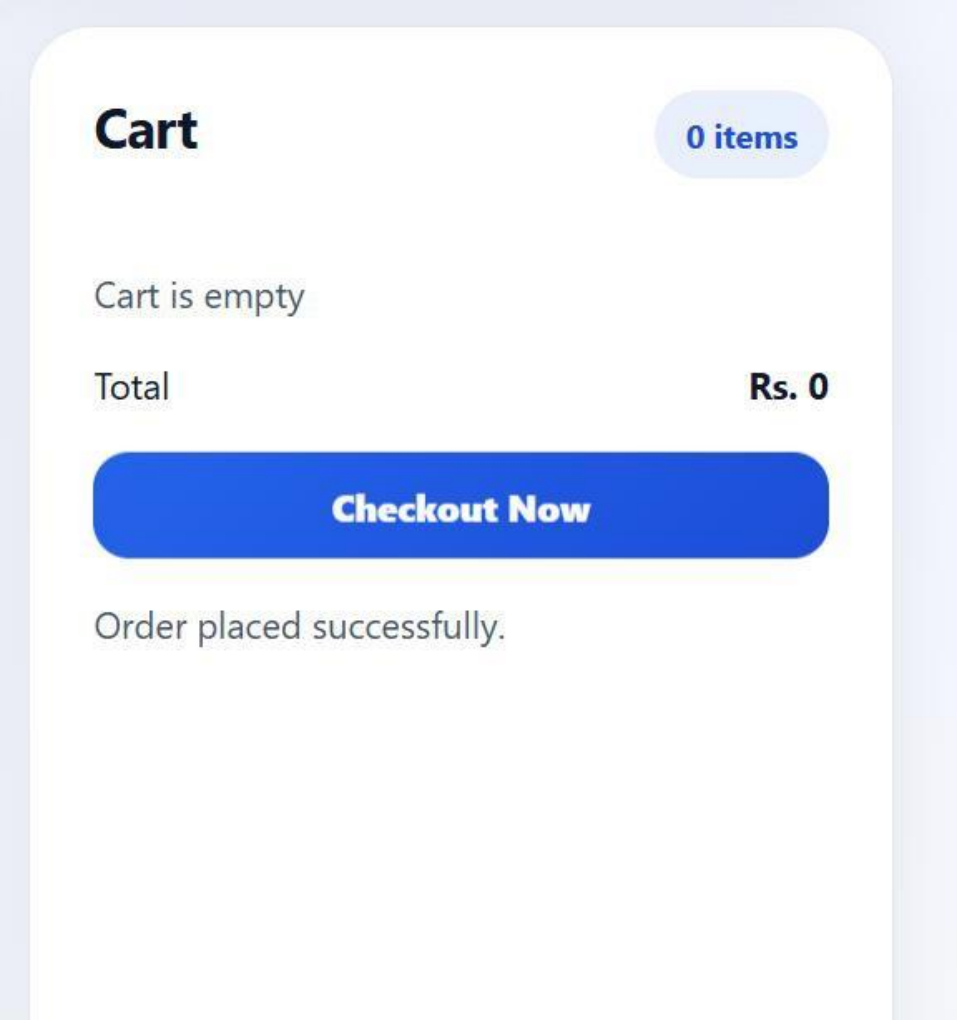
- **Inventory Tracking**: Each card displays a "Stock" badge that is dynamically updated after every successful "Add To Cart" operation .
- **Input Validation**: The quantity selector (Qty) defaults to 1, and the backend validates that any user input is a positive integer before processing the transaction .
- **Information Architecture**: Detailed specifications (e.g., "8GB RAM," "10000mAh") are displayed directly on the card to assist users in making informed purchasing decisions without navigating to a separate page

iii) Cart



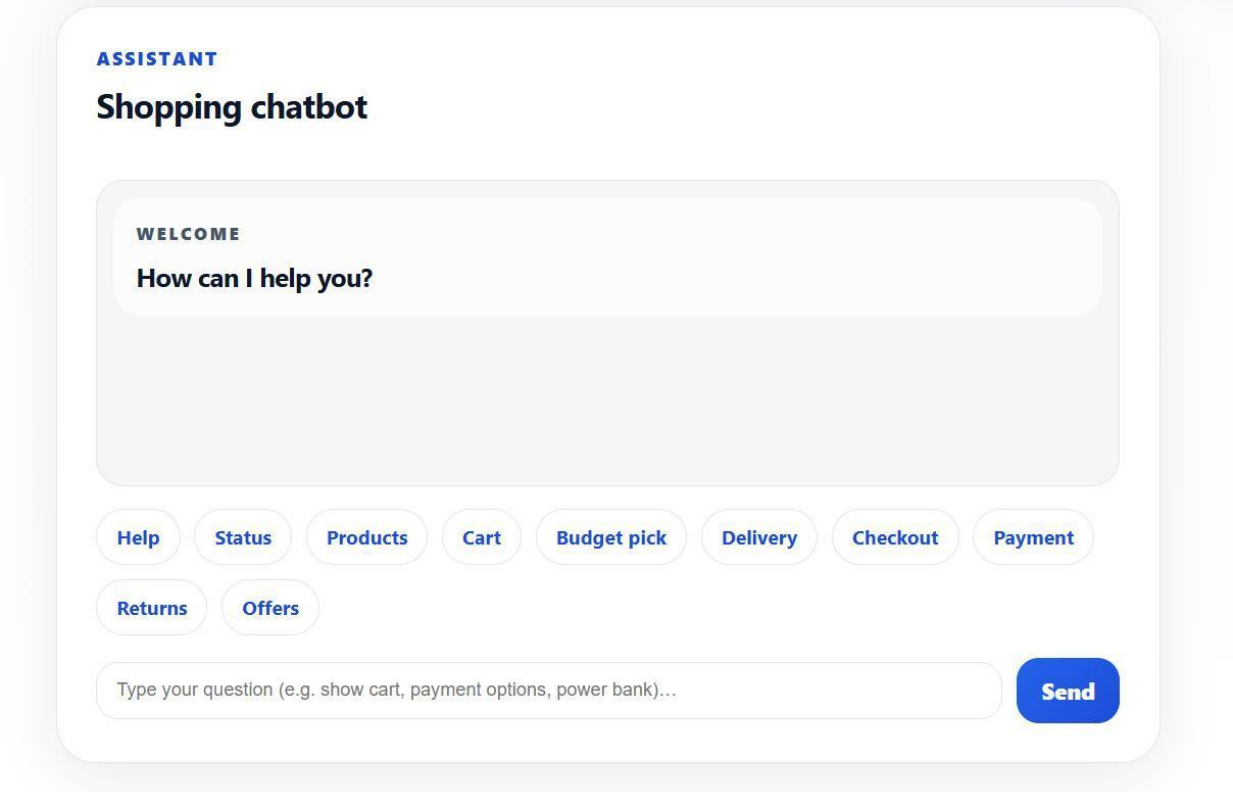
- **State Synchronization**: The cart list is rendered by mapping the current state object in app.js, ensuring that the "Total" accurately reflects the sum of price * quantity for all selected items .
- **Currency Formatting**: Prices are displayed with "Rs." prefixes to maintain a consistent local currency format for the user .
- **Checkout Readiness**: The "Checkout Now" button remains active only when items are present, linking directly to the order processing logic.

iv) Checkout



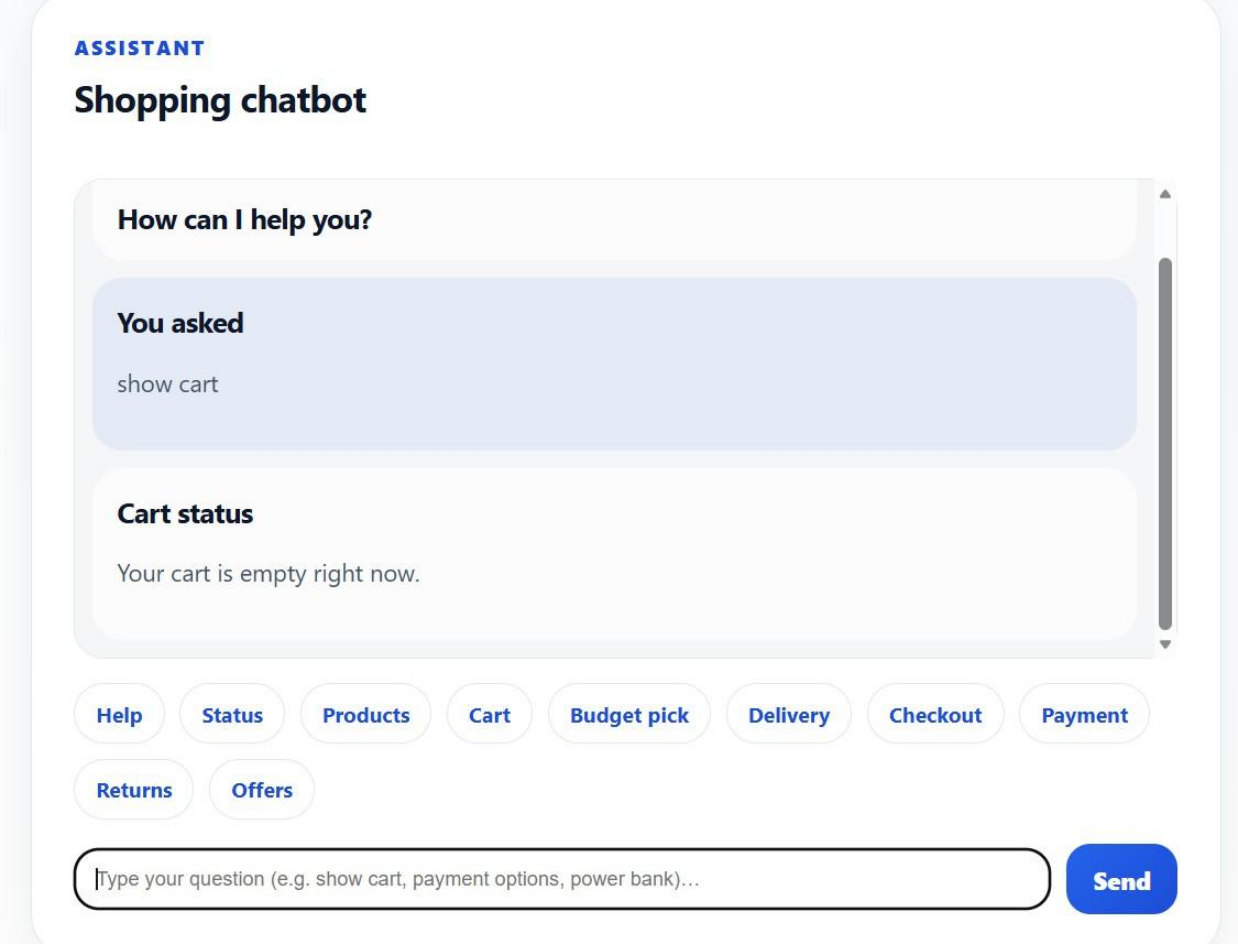
- **Transaction Finalization**: This view confirms the execution of the /checkout route, which clears the cart array in data.json and prepares the system for a new session .
- **Zero-State Feedback**: Upon successful checkout, the UI displays "Cart is empty," providing the user with visual confirmation that the order has been submitted.

v) Chatbot Quick response



- **Action Chips**: The buttons (Help, Status, Products, etc.) serve as "Quick Actions," allowing users to trigger complex queries with a single click rather than typing full sentences .
- **Intent Mapping**: Clicking a chip sends a specific "action" parameter to the /chatbot endpoint, which the backend uses to return a deterministic, structured response.

vi) Chatbot Typed Query Response



 Natural Language Parsing: The system captures raw text input (e.g., "show cart") and uses keyword-based recognition to identify the user's intent.

 Contextual Responses: The assistant provides dynamic feedback, such as stating "Your cart is empty right now," by querying the current status of the user's session before responding

5. Conclusions

This project demonstrates the successful development of a web-based natural language assisted e-commerce platform from an initial CLI prototype. It validates full-stack concepts by integrating a structured frontend, REST-style backend, and persistent data layer into one coherent system.

The implemented architecture supports practical workflows such as product discovery, stock-aware cart updates, checkout processing, and user guidance via chatbot. The project also emphasizes maintainability through modular code organization and clear responsibility boundaries.

From an academic perspective, the project satisfies objectives related to software design, implementation, testing, interface development, and system evaluation. It also provides a strong foundation for future production-oriented enhancements.

6. Further development or research

Recommended future scope:

- Integrate relational database (MySQL/PostgreSQL) for transactional integrity.
- Add authentication and role-based authorization.
- Support user-specific carts and order history.
- Add remove/update quantity controls in cart.
- Implement payment gateway and invoice generation.
- Add product search, filters, and category pages.
- Build admin panel for product and inventory management.
- Introduce automated tests (unit + API + UI).
- Deploy to cloud with CI/CD pipeline.
- Enhance chatbot using NLP model and contextual memory.
- Add analytics dashboard for sales and product trends.
- Apply caching strategy for higher performance at scale.

Research directions:

- Intent classification for multilingual shopping queries.
- Hybrid rule-based + ML chatbot architecture.
- Real-time stock synchronization with WebSocket updates.
- Personalized recommendation engine.

7. References

- Python Software Foundation. *Python Documentation*. https://docs.python.org/3/
- Flask Documentation. *Flask Web Development Framework*. https://flask.palletsprojects.com/
- MDN Web Docs. *HTML, CSS, JavaScript, Fetch API*. https://developer.mozilla.org/
- Fielding, R. *Architectural Styles and the Design of Network-based Software Architectures* (REST principles).
- JSON.org. *Introducing JSON*. https://www.json.org/
- W3C. *Web Standards Documentation*. https://www.w3.org/

8. Appendix

- Complete source file list:
    - app.py
    - templates/index.html
    - static/app.js
    - static/styles.css
    - data.json
    - main.py (optional CLI)
- API request/response samples:
    - POST /add payload and response
    - GET /checkout response
    - POST /chatbot action/message examples
- Test case checklist and outcomes
- Setup and run instructions
- Known issues and workaround notes
- Additional UI snapshots
- Change log from CLI version to website version

8.1 Weekly report

- Weekly project report for week commencing-02 March

02-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1

For week commencing 02 March 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: Alena Latheesh
Register Number: **23BCAICD015**
WPPR: 1
Internal Guide's Name: Mr. Praveen

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Develop backend using Flask
- Implement product-related APIs
- Connect CLI with backend
- Implement NLP command detection

Progress/Achievements for the week:

- DevelopedFlask backend with REST APIs
- Implemented endpoints:
  /products
  /products/<id>
- CreatedJSON file for product database
- IntegratedCLI with backend using HTTP requests
- Implemented basic NLP keyword-based command parsing

1

02-03-2026

Future Work Plans:

- Develop cart functionality
- Implement add-to-cart with validation
- Improve NLP command recognition
- Start checkout module

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

- Weekly project report for week commencing- 09 March

09-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 2

For week commencing 09 March 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: Alena Latheesh
Register Number: **23BCAICD015**
WPPR: 2
Internal Guide's Name: Mr. Praveen

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Develop cart functionality backend
- Implement add-to-cart with validation
- Build CLI cart viewing flow

Progress/Achievements for the week:

- Implemented the /cart and /add POST endpoints in the Flask backend (app.py) to manage cart state.
- Added backend validation logic to ensure requested quantities are positive integers and that sufficient product stock exists in data.json before adding to the cart.
- Updated the CLI client (main.py) with a cart_flow function that retrieves cart data, maps prices to calculate subtotals, and dynamically displays the final total amount.
- Created helper functions (build_cart_summary and build_overview) to consistently calculate item counts and total costs.

1

09-03-2026

Future Work Plans:

- Develop the final checkout backend logic.
- Implement CLI checkout confirmation flow.
- Expand the NLP backend logic to handle more complex user queries.

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

2

- Weekly project report for week commencing- 16 March

16-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 3

For week commencing 16 March 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: Alena Latheesh
Register Number: **23BCAICD015**
WPPR: 3
Internal Guide's Name: Mr. Praveen

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Develop the final checkout backend logic.
- Implement CLI checkout confirmation flow.
- Expand the NLP backend logic to handle more complex user queries.

Progress/Achievements for the week:

- Built the /checkout endpoint in app.py to process orders, return success messages, and clear the cart state.
- Implemented the user confirmation prompt inside the CLI cart_flow() to finalize the checkout process.
- Built the user profile viewing functionality (profile_flow()) within the CLI.
- Started structuring the core NLP intent-matching engine (build_chat_response) to handle keyword variations for statuses, best values, and delivery help.

1

16-03-2026

Future Work Plans:

- Finalize all chatbot text responses (payment, returns, contact help).
- Conduct end-to-end testing of the CLI client.
- Prepare repository documentation.

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

- Weekly project report for week commencing- 23 March

23-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 4

For week commencing 23 March 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: Alena Latheesh
Register Number: **23BCAICD015**
WPPR: 4
Internal Guide's Name: Mr. Praveen

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Finalize all chatbot text responses (payment, returns, contact help).
- Conduct end-to-end testing of the CLI client.
- Prepare repository documentation.

Progress/Achievements for the week:

- Completed the NLP build_chat_response logic, allowing the system to recognize intents like "payment options", "return policy", and "offers".
- Structured the chatbot text outputs into a clean, reusable format featuring titles, messages, and itemized lists for clear CLI reading.
- Executed extensive manual end-to-end testing, simulating various user NLP inputs in the CLI to ensure backend state (data.json) remains synchronized.
- Drafted the initial project documentation (README.md), outlining local setup instructions and the required CLI execution commands.

1

23-03-2026

Future Work Plans:

- Perform final code reviews and optimizations for the CLI build.
- Configure environment variables and .gitignore file.
- Push the final CLI/Backend codebase to the GitHub repository.

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

- Weekly project report for week commencing- 30 March

30-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 5

For week commencing 30 March 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: Alena Latheesh
Register Number: **23BCAICD015**
WPPR: 5
Internal Guide's Name: Mr. Praveen

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Perform final code reviews and optimizations for the CLI build.
- Configure environment variables and .gitignore file.
- Push the final CLI/Backend codebase to the GitHub repository.

Progress/Achievements for the week:

- Conducted final code walkthroughs, ensuring all Flask endpoints and NLP parsing logic function without crashing on unexpected inputs.
- Created a .gitignore file to prevent tracking of unnecessary files such as ecom_env/, __pycache__/, .pyc, and .zip archives.
- Finalized the README.md to cleanly document API routes and project structure.
- Successfully initialized Git, committed the final version of the CLI application, and pushed the repository to GitHub.

30-03-2026

Future Work Plans:

- Begin researching and planning a web-based GUI (HTML/CSS/JS) to replace the CLI interface.
- Plan API updates to serve front-end dashboard data.

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

- Weekly report for week commencing- 13 April

13-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 6

For week commencing 13 April 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: Alena Latheesh
Register Number: **23BCAICD015**
WPPR: 6
Internal Guide's Name: Mr. Praveen

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Develop a web-based GUI (Dashboard) using HTML/CSS.
- Integrate the chatbot directly into the web dashboard interface.
- Create a dedicated endpoint for the web front-end.

Progress/Achievements for the week:

- Created the main index route (/) in Flask using render_template to serve a web-based dashboard.
- Built the front-end layout (index.html and styles.css) featuring product cards with quantity selection, a cart summary, and a profile snapshot.
- Integrated a right-side interactive chatbot drawer directly into the web dashboard, allowing users to interact with the NLP engine visually.
- Added the /dashboard-data API endpoint to supply real-time JSON summaries of products and cart state to the web client.
- Successfully completed all front-end integrations on April 17th.

13-04-2026

Future Work Plans:

- Prepare project presentation and live demonstration materials for the final evaluation.

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**

- Weekly report for week commencing- 20 April

20-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 7

For week commencing 20 April 2026

Programme: BCA (Artificial Intelligence, Cloud Computing, and DevOps)

Student Name: Alena Latheesh
Register Number: **23BCAICD015**
WPPR: 7
Internal Guide's Name: Mr. Praveen

MAJOR PROJECT Title:

Design and Development of Natural Language Interface for E-commerce Infrastructure

Targets set for the week:

- Draft the comprehensive final project report.
- Prepare project presentation slides and demonstration materials.
- Conduct a final review of the entire project repository and documentation.

Progress/Achievements for the week:

- Documented the complete system architecture, including the Flask backend, NLP intent-matching flow, and front-end dashboard integrations
- Finalized the project report document formatting according to the university guidelines.
- Verified that the README.md and repository structure are completely ready for final evaluation.

20-04-2026

Future Work Plans:

- Submit the final project documentation and codebase.
- Prepare for the final project presentation and viva voce/evaluation.

Implementation shown: ☐ Yes ☐ No

Remarks by the Internal Guide:

_____

_____

_____

Signature of the student **Signature of the Internal Guide**